

Этот пакет — это **HTTP-клиент (обертка)** на языке Go, написанный в стиле **Fluent Interface** (цепочки вызовов). Он предназначен для упрощения выполнения HTTP-запросов, автоматической сериализации/десериализации (JSON, XML), логирования и обработки повторных попыток (retries).

Что делает этот пакет?

1. **Удобство (Fluent API):** Позволяет писать запросы в одну строку: `httpq.NewRpc().Post().SetUrl("...").Json().SetBody(data).Do(ctx)`.
 2. **Дженерики:** Использует `ResponseModel[T]`, что позволяет сразу получить готовую структуру данных из ответа.
 3. **Повторы (Retries):** Поддерживает настраиваемую политику повторов (экспоненциальная задержка, фильтрация по статус-кодам).
 4. **Логирование:** Встроенная интеграция с `slog` для отладки запросов и ответов.
 5. **Контексты:** Полная поддержка `context.Context` для отмены запросов и таймаутов.
-

Есть ли в нем ошибки?

Код написан достаточно чисто, но есть несколько **критических архитектурных проблем** и потенциальных багов, которые могут проявиться под нагрузкой или в специфических сценариях.

1. Проблема конкурентности (Thread Safety) — Критично

Метод `NewRpc` инициализирует клиент: `http.DefaultClient`.

- **Ошибка:** Когда вы вызываете `SetTimeout` или `SetTransport`, вы **меняете глобальный `http.DefaultClient`**.
- **Последствие:** Если одна часть вашей программы вызовет `SetTimeout(1 * time.Second)`, а другая — `SetTimeout(30 * time.Second)`, они будут перезаписывать настройки друг друга. Это может привести к непредсказуемому поведению всей программы.
- **Как исправить:** В `NewRpc` создавайте новый экземпляр клиента: `client: &http.Client{}`.

2. Утечка ресурсов (Body leak) — Средне

В функции `doOnce` есть код:

Go

```

resp, err := r.client.Do(req)
if err != nil { ... return }
defer resp.Body.Close()

var bodyBytes []byte
if resp.Body != http.NoBody {
    bodyBytes, _ = io.ReadAll(resp.Body)
}

```

- **Ошибка:** В Go, если `resp` не равен `nil`, тело **обязательно** нужно вычитать до конца и закрыть, даже если статус-код плохой или тело пустое. Хотя `defer` закрывает тело, `io.ReadAll` может не выполниться, если `resp.Body == http.NoBody`. Однако `http.NoBody` — это не `nil`.
- **Рекомендация:** Безопаснее всегда вызывать `io.Copy(io.Discard, resp.Body)` перед закрытием, чтобы переиспользовать TCP-соединение (Keep-Alive).

3. Ошибка логики в `isRetryableStatus`

Go

```

if code == 0 { return true }

```

- **Проблема:** `code == 0` обычно означает сетевую ошибку (DNS, отказ в соединении). Это правильно ретраить. Однако, если `policy.RetryStatusCodes` заполнен, код проверит только его.
- **Логика:** Если пользователь передал пустой список кодов, ретраев вообще не будет, даже при `code == 0`.

4. Неэффективный `MultiPart`

Go

```

bts, errMarshal := json.Marshal(r.body)
_ = json.Unmarshal(bts, &values)

```

- **Ошибка:** Это очень странный и медленный способ конвертации структуры в карту (Map). Он требует двойного копирования данных в памяти.
- **Последствие:** Огромные затраты CPU и памяти на ровном месте.

5. Потенциальный Panic в MultiPart

Go

```
if errMarshal != nil {  
    panic(errMarshal)  
}
```

- **Ошибка:** Использование panic в библиотечном коде — плохая практика. Если во входных данных окажется что-то, что нельзя маршализовать в JSON (например, канал или функция), всё приложение упадет. Лучше возвращать error.

Резюме

Вердикт: Пакет удобен для небольших утилит или внутренних инструментов.

Что нужно исправить перед использованием в продакшене:

1. **Обязательно** перестать использовать http.DefaultClient внутри NewRpc.
2. Заменить panic на возврат ошибки в блоке MultiPart.
3. Проверить логику isRetryableStatus, чтобы сетевые ошибки (код 0) всегда приводили к ретрау, если включена политика ретраев.

Хотите, чтобы я помог исправить проблему с DefaultClient или переписал блок MultiPart на более эффективный?